

Лабораторна робота № 1

Вхідний аналіз набору даних

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи · **Максимальна оцінка:** 4 бали

Мета роботи

Пригадати роботу в Jupyter Notebook, бібліотеці `pandas` та побудову графіків — і зробити це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для вас.

Ця робота виконується **до першої лекції**. Нічого нового знати не потрібно: усе, що знадобиться, ви вже проходили в попередніх курсах. Усі поняття, яких ви тут торкнетесь мимохідь, отримують назви й пояснення на лекціях 3 і 4.

Дані

Файл `20171129_FINAL_FINAL.xlsx`, аркуш `Sheet1` — спостереження космічної погоди за 28 серпня — 22 вересня 2017 року, зібрані з трьох різних джерел.

Файл видається **як є**, рівно в тому вигляді, у якому його колись вивантажили. Ніхто не приводив його до ладу, і саме в цьому сенс: справжні дані здебільшого надходять саме такими.

Що ви маєте зробити

Етап	Зміст	Де
1	Розібратися, що взагалі лежить у файлі	аудиторно
2	Виділити три таблиці й привести їх до робочого вигляду	аудиторно
3	Скласти паспорт даних: що є, чого бракує, що виглядає дивно	аудиторно
4	Об'єднати три таблиці в одну	аудиторно
5	Побудувати п'ять графіків, кожен — відповідь на питання	аудиторно + самостійно
6	Порівняти два графіки одних і тих самих даних	самостійно
7	Записати три спостереження про дані	самостійно

Етап	Зміст	Де
8	Декларація про використання інструментів ШІ	самостійно

Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Ноутбук має виконуватися згори донизу на чистому ядрі** (Kernel → Restart & Run All) без помилок — це перевіряється першою дією на захисті.
2. Файл `master_hourly.csv` — об'єднана таблиця з етапу 4.
3. Експорт ноутбука у HTML або PDF.
4. Заповнена декларація (етап 8). Без неї фінальна самоперевірка не проходить.

Критерії оцінювання

Бали	Що досягнуто
0	Дані не опрацьовано, графіки не побудовано або побудовано з грубими помилками
1	Таблиці прочитано й кілька графіків побудовано, але типи графіків не відповідають даним, дивних значень не помічено
2	Таблиці розібрано й об'єднано, графіки коректні та підписані, але висновків під ними немає або вони формальні
4	Усе вище виконано, знайдено й пояснено дивні значення, під кожним графіком є змістовний висновок про дані

Головне про оцінку: бали ставляться не за наявність графіка, а за речення під ним. Код, який нічого не пояснює, коштує двох балів з чотирьох.

Нагадування про Jupyter

Якщо давно не працювали — три речі, які знадобляться сьогодні:

- `Shift+Enter` виконує клітинку, `Esc A` / `Esc B` додає клітинку вище / нижче, `Esc M` перетворює клітинку на текстову (Markdown), `Esc Y` — назад на код.
- **Порядок виконання має значення.** Якщо ви правили клітинки в довільному порядку, результат може залежати від історії запусків. Перед здачею обов'язково **Kernel → Restart & Run All** — так ви побачите ноутбук очима викладача.
- Текстові клітинки — не прикраса. Висновки пишуться в них, а не в коментарях до коду.

Крок 0. Ваші дані та варіант

Заповніть три змінні й виконайте обидві клітинки. Друга надрукує **ваше індивідуальне завдання** — воно відрізняється від завдання сусіда.

```
In [1]: ПІБ = 'Бугай Софія'          # напр. 'Шевченко Тарас'
ГРУПА = 'ШІ-34'                    # напр. 'КН-31'
ВАРІАНТ = 2                        # ваш номер за списком групи

assert ПІБ and ГРУПА and ВАРІАНТ, 'Заповніть ПІБ, ГРУПУ та ВАРІАНТ'
```

```
In [2]: # --- клітинку не редагувати: вона видає ваше персональне завдання -----
ПОТОКИ = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']

ПАРИ = [('швидкість вітру', 'густина вітру'),
         ('швидкість вітру', 'потік P>10'),
         ('густина вітру', 'потік P>1'),
         ('радіопотік F10.7', 'потік P>10'),
         ('радіопотік F10.7', 'швидкість вітру'),
         ('потік E>0.8', 'потік P>10')]

АГРЕГАТИ = ['середнє', 'максимум', 'медіана', 'мінімум']

ПИТАННЯ = [
    'Скільки годин за все вікно ваш потік перевищував своє власне медіанне значення',
    'У яку добу середнє значення вашого потоку було найбільшим? А найменшим?',
    'О котрій годині доби потік E>0.8 у середньому найбільший? Побудуйте графік сер',
    'Яка частка рядків об'єднаної таблиці має пропуск хоча б в одному стовпці?',
    'У який день радіопотік F10.7 був найбільшим і чи збігається цей день з піком в',
    'Порівняйте медіану вашого потоку за перший і за другий тиждень спостережень. У',
    'Скільки годин поспіль тривала найдовша неперервна серія «активних» годин (груп',
    'Який зі стовпців об'єднаної таблиці має найбільший розкид відносно власного се',
    'Яку частку сумарного значення вашого потоку за все вікно дали 24 години навкол',
    'Скільки різних діб потрапило у групу «активні» і скільки – у групу «дуже актив']

i = int(ВАРІАНТ)
завдання = {
    'ваш потік': ПОТОКИ[(i * 3 + 1) % 8],
    'ваша пара для графіка 3': ' та '.join(ПАРИ[(i * 5 + 2) % 6]),
    'ваш агрегат для етапу 4': АГРЕГАТИ[(i * 5 + 1) % 4],
    'ваше додаткове питання': ПИТАННЯ[(i * 11 + 5) % 10],
}

print('ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · %s, %s, варіант %d' % (ПІБ, ГРУПА, i))
print('=' * 78)
for k, v in завдання.items():
    print('%-28s %s' % (k + ': ', v))
```

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · Бугай Софія, ШІ-34, варіант 2

=====

ваш потік: E>2.0
 ваша пара для графіка 3: швидкість вітру та густина вітру
 ваш агрегат для етапу 4: мінімум
 ваше додаткове питання: Який зі стовпців об'єднаної таблиці має найбільший розкид відносно власного середнього?

Далі за текстом «**ваш потік**» означає стовпець, названий у першому рядку завдання.

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', 40)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                    'axes.spines.right': False, 'font.size': 9})

ФАЙЛ = '20171129_FINAL_FINAL.xlsx'
```

Етап 1. Що взагалі лежить у файлі

Перше правило роботи з незнайомими даними: спершу подивитися, потім рахувати.

Завдання 1.1

Спробуйте прочитати аркуш `Sheet1` звичайним способом — `pd.read_excel(ФАЙЛ)` — і виведіть перші рядки та типи стовпців.

Очікуваний результат: результат буде дивним. Подивіться на назви стовпців і на те, які там типи даних. Поясніть у наступній клітинці, чому так вийшло.

```
In [4]: # ВАШ КОД
dp = pd.read_excel(ФАЙЛ)
print(dp.head(25))
```

```

      Unnamed: 0      ftp://ftp.swpc.noaa.gov/pub/lists/particle/ Unnamed: 2 Unname
d: 3 Unnamed: 4 Unnamed: 5 Unnamed: 6 Unnamed: 7 Unnamed: 8 Unnamed: 9 Unnamed: 10 U
nnamed: 11 Unnamed: 12 \
0      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
1      NaN      :Data_list: 20170901_Gs_part_5m.txt      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
2      NaN      :Created: 2017 Sep 02 0011 UTC      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
3      NaN # Prepared by the U.S. Dept. of Commerce, NOAA...      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
4      NaN # Please send comments and suggestions to SWPC...      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
5      NaN      #      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
6      NaN      # Label: P > 1 = Particles at >1 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
7      NaN      # Label: P > 5 = Particles at >5 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
8      NaN      # Label: P >10 = Particles at >10 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
9      NaN      # Label: P >30 = Particles at >30 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
10     NaN      # Label: P >50 = Particles at >50 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
11     NaN      # Label: P>100 = Particles at >100 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
12     NaN      # Label: E>0.8 = Electrons at >0.8 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
13     NaN      # Label: E>2.0 = Electrons at >2.0 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
14     NaN      # Label: E>4.0 = Electrons at >4.0 Mev      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
15     NaN      # Units: Particles = Protons/cm2-s-sr      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
16     NaN      # Units: Electrons = Electrons/cm2-s-sr      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN
17     NaN      # Source: GOES-15      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN

```

	Unnamed: 13	Unnamed: 14	Unnamed: 15			Unnamed: 22	Unnamed: 23
0	NaN	NaN	NaN				
1	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN				
3	NaN	NaN	NaN				
4	NaN	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN				
6	NaN	NaN	NaN	NaN	NaN	NaN	
7	NaN	NaN	NaN				
8	NaN	NaN	NaN				
9	NaN	NaN	NaN	NaN	NaN	NaN	
10	NaN	NaN	NaN				
11	NaN	NaN	NaN	NaN	NaN	NaN	
12	NaN	NaN	NaN				
13	NaN	NaN	NaN	NaN	NaN	NaN	
14	NaN	NaN	NaN				
15	NaN	NaN	NaN	NaN	NaN	NaN	
16	NaN	NaN	NaN				
17	NaN	NaN	NaN	NaN	NaN	NaN	
18	NaN	NaN	NaN				
19	NaN	NaN	NaN	NaN	NaN	NaN	
20	NaN	NaN	NaN				
21	NaN	NaN	NaN	NaN	NaN	NaN	
22	NaN	NaN	NaN				
23	NaN	NaN	NaN	NaN	NaN	NaN	
24	NaN	NaN	NaN				
25	NaN	NaN	NaN	NaN	NaN	NaN	
26	NaN	NaN	NaN				
27	NaN	NaN	NaN	NaN	NaN	NaN	
28	NaN	NaN	NaN				
29	NaN	NaN	NaN	NaN	NaN	NaN	
30	NaN	NaN	NaN				
31	NaN	NaN	NaN	NaN	NaN	NaN	
32	NaN	NaN	NaN				
33	NaN	NaN	NaN	NaN	NaN	NaN	
34	NaN	NaN	NaN				
35	NaN	NaN	NaN	NaN	NaN	NaN	
36	NaN	NaN	NaN				
37	NaN	NaN	NaN	NaN	NaN	NaN	
38	NaN	NaN	NaN				
39	NaN	NaN	NaN	NaN	NaN	NaN	
40	NaN	NaN	NaN				
41	NaN	NaN	NaN	NaN	NaN	NaN	
42	NaN	NaN	NaN				
43	NaN	NaN	NaN	NaN	NaN	NaN	
44	NaN	NaN	NaN				
45	NaN	NaN	NaN	NaN	NaN	NaN	
46	NaN	NaN	NaN				
47	NaN	NaN	NaN	NaN	NaN	NaN	
48	NaN	NaN	NaN				
49	NaN	NaN	NaN	NaN	NaN	NaN	
50	NaN	NaN	NaN				
51	NaN	NaN	NaN	NaN	NaN	NaN	
52	NaN	NaN	NaN				
53	NaN	NaN	NaN	NaN	NaN	NaN	
54	NaN	NaN	NaN				
55	NaN	NaN	NaN	NaN	NaN	NaN	
56	NaN	NaN	NaN				
57	NaN	NaN	NaN	NaN	NaN	NaN	
58	NaN	NaN	NaN				
59	NaN	NaN	NaN	NaN	NaN	NaN	
60	NaN	NaN	NaN				
61	NaN	NaN	NaN	NaN	NaN	NaN	
62	NaN	NaN	NaN				
63	NaN	NaN	NaN	NaN	NaN	NaN	
64	NaN	NaN	NaN				
65	NaN	NaN	NaN	NaN	NaN	NaN	
66	NaN	NaN	NaN				
67	NaN	NaN	NaN	NaN	NaN	NaN	
68	NaN	NaN	NaN				
69	NaN	NaN	NaN	NaN	NaN	NaN	
70	NaN	NaN	NaN				
71	NaN	NaN	NaN	NaN	NaN	NaN	
72	NaN	NaN	NaN				
73	NaN	NaN	NaN	NaN	NaN	NaN	
74	NaN	NaN	NaN				
75	NaN	NaN	NaN	NaN	NaN	NaN	
76	NaN	NaN	NaN				
77	NaN	NaN	NaN	NaN	NaN	NaN	
78	NaN	NaN	NaN				
79	NaN	NaN	NaN	NaN	NaN	NaN	
80	NaN	NaN	NaN				
81	NaN	NaN	NaN	NaN	NaN	NaN	
82	NaN	NaN	NaN				
83	NaN	NaN	NaN	NaN	NaN	NaN	
84	NaN	NaN	NaN				
85	NaN	NaN	NaN	NaN	NaN	NaN	
86	NaN	NaN	NaN				
87	NaN	NaN	NaN	NaN	NaN	NaN	
88	NaN	NaN	NaN				
89	NaN	NaN	NaN	NaN	NaN	NaN	
90	NaN	NaN	NaN				
91	NaN	NaN	NaN	NaN	NaN	NaN	
92	NaN	NaN	NaN				
93	NaN	NaN	NaN	NaN	NaN	NaN	
94	NaN	NaN	NaN				
95	NaN	NaN	NaN	NaN	NaN	NaN	
96	NaN	NaN	NaN				
97	NaN	NaN	NaN	NaN	NaN	NaN	
98	NaN	NaN	NaN				
99	NaN	NaN	NaN	NaN	NaN	NaN	

10	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
11	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
12	NaN	NaN	NaN	ftp://ftp.swpc.noaa.gov/pub/indices/old_ind		
ice...	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
13	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
14	NaN	NaN	NaN	Product: Daily Solar Data		quar_D
SD.txt	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
15	NaN	NaN	NaN		:Issued: 0825 UT 04 Oc	
t 2017	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
16	NaN	NaN	NaN			
#	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
17	NaN	NaN	NaN	# Prepared by the U.S. Dept. of Commerce,		
NOA...	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
18	NaN	NaN	NaN	# Please send comments and suggestions to		
SWP...	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
19	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
20	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
21	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
22	NaN	NaN	NaN		Quarterly Daily Sola	
r Data	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
23	NaN	NaN	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					
24	E>0.8	E>2.0	NaN			
NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN					

Unnamed: 25		http://umtof.umd.edu/pm/crn/		Unnamed: 27 Unnam	
ed: 28 Unnamed: 29		Unnamed: 30		Unnamed: 31	
0	NaN			NaN	NaN
NaN	NaN	NaN	NaN		
1	NaN		SOHO CELIAS Proton Monitor		NaN
NaN	NaN	NaN	NaN		
2	NaN	Solar Wind Parameters for Carrington Rotation ...			NaN
NaN	NaN	NaN	NaN		
3	NaN		Proton speed [km/sec]		NaN
NaN	NaN	NaN	NaN		

4	NaN	Proton density [protons per cubic centimeter]	NaN
NaN	NaN	NaN	NaN
5	NaN		NaN
NaN	NaN	NaN	NaN
6	NaN	Measurement time (Year, Month, Day of Month, D...	NaN
NaN	NaN	NaN	NaN
7	NaN		NaN
NaN	NaN	NaN	NaN
8	NaN		NaN
NaN	NaN	NaN	NaN
9	NaN		NaN
NaN	NaN	NaN	NaN
10	NaN		NaN
NaN	NaN	NaN	NaN
11	NaN		NaN
NaN	NaN	NaN	NaN
12	NaN		NaN
NaN	NaN	NaN	NaN
13	NaN		NaN
NaN	NaN	NaN	NaN
14	NaN		NaN
NaN	NaN	NaN	NaN
15	NaN		NaN
NaN	NaN	NaN	NaN
16	NaN		NaN
NaN	NaN	NaN	NaN
17	NaN		NaN
NaN	NaN	NaN	NaN
18	NaN		NaN
NaN	NaN	NaN	NaN
19	NaN		NaN
NaN	NaN	NaN	NaN
20	NaN		NaN
NaN	NaN	NaN	NaN
21	NaN		NaN
NaN	NaN	NaN	NaN
22	NaN		NaN
NaN	NaN	NaN	NaN
23	NaN		NaN
NaN	NaN	NaN	NaN
24	NaN		NaN
NaN	NaN	NaN	NaN

Відповідь 1.1: Функція, без додаткових аргументів, зчитує кожну клітинку, взявши за назви стовпців значення клітинки відповідного стовпця у першому ряді. Усі пусті клітинки функція зчитує як NaN, а пусті заголовки - як Unnamed : index. Без специфікації, назви рядків функція позначає як послідовний індекс (починаючи з 0). Максимальна кількість стовпців визначається тим, у якому останньому стовпці з'являються хоч якісь дані.

Завдання 1.2

Прочитайте аркуш ще раз, але **без заголовка**: `pd.read_excel(ФАЙЛ, header=None)` .
Виведіть його розмір і подивіться на перші 30 рядків.

```
In [5]: # ВАШ КОД
RAW = pd.read_excel(ФАЙЛ, header=None)
print("Розмір аркуша: ", RAW.size)
print("Довжина аркуша: ", RAW.shape[0])
print("Ширина аркуша: ", RAW.shape[1])
print("-----")
print(RAW.head(30))
```

Розмір аркуша: 231232

Довжина аркуша: 7226

Ширина аркуша: 32

```

-----
      0                                     1   2   3   4           5
6      7      8      9      10     11     12     13     14  15  \
0  NaN      ftp://ftp.swpc.noaa.gov/pub/lists/particle/  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
1  NaN      NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
2  NaN      :Data_list: 20170901_Gs_part_5m.txt  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
3  NaN      :Created: 2017 Sep 02 0011 UTC  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
4  NaN  # Prepared by the U.S. Dept. of Commerce, NOAA...  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
5  NaN  # Please send comments and suggestions to SWPC...  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
6  NaN      #  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
7  NaN      # Label: P > 1 = Particles at >1 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
8  NaN      # Label: P > 5 = Particles at >5 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
9  NaN      # Label: P >10 = Particles at >10 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
10 NaN      # Label: P >30 = Particles at >30 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
11 NaN      # Label: P >50 = Particles at >50 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
12 NaN      # Label: P>100 = Particles at >100 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
13 NaN      # Label: E>0.8 = Electrons at >0.8 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
14 NaN      # Label: E>2.0 = Electrons at >2.0 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
15 NaN      # Label: E>4.0 = Electrons at >4.0 Mev  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
16 NaN      # Units: Particles = Protons/cm2-s-sr  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
17 NaN      # Units: Electrons = Electrons/cm2-s-sr  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
18 NaN      # Source: GOES-15  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
19 NaN      # Location: W135  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
20 NaN      # Missing data: -1.00e+05  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
21 NaN      NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
22 NaN  5-minute GOES-15 Solar Particle and Electron ...  NaN  NaN  NaN  NaN
NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
23 NaN      NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
Seconds      NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN
24 NaN      # UT C D ate Time  Julian
of the      NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN

```

25	NaN						# YR	MO	DA	HHMM	Day
Day	P > 1	P > 5	P > 10	P > 30	P > 50	P > 100	E > 0.8	E > 2.0	NaN		
26	NaN						2017	8	28	0	57993
0	12	0.26	0.18	0.114	0.104	0.0793	23600	1180	NaN		
27	NaN						2017	8	28	5	57993
300	19.4	0.69	0.314	0.112	0.0612	0.0251	23100	1160	NaN		
28	NaN						2017	8	28	10	57993
600	8.48	0.168	0.13	0.0717	0.0614	0.0304	22200	1090	NaN		
29	NaN						2017	8	28	15	57993
900	15.4	0.566	0.198	0.0926	0.0568	0.0251	20400	997	NaN		
							16	17	18		19
20	21	22	23	24	25					26	27 2
8	29		30		31						
0							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN		http://umtof.umd.edu/pm/crn/				NaN N
aN	NaN		NaN		NaN						
1							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
2							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN		SOHO CELIAS Proton Monitor				NaN N
aN	NaN		NaN		NaN						
3							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN	Solar Wind Parameters for Carrington Rotation ...					NaN N
aN	NaN		NaN		NaN						
4							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN		Proton speed [km/sec]				NaN N
aN	NaN		NaN		NaN						
5							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN	Proton density [protons per cubic centimeter]					NaN N
aN	NaN		NaN		NaN						
6							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
7							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN	Measurement time (Year, Month, Day of Month, D...					NaN N
aN	NaN		NaN		NaN						
8							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
9							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
10							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
11							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
12							NaN	NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						
13	ftp://ftp.swpc.noaa.gov/pub/indices/old_indice...							NaN	NaN		NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN N
aN	NaN		NaN		NaN						

14						NaN	NaN	NaN		NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
15	Product: Daily Solar Data					quar_DSD.txt	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
16						:Issued: 0825 UT 04 Oct 2017	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
17						#	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
18	#	Prepared by the U.S. Dept. of Commerce, NOAA...					NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
19	#	Please send comments and suggestions to SWP...					NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
20						NaN	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
21						NaN	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
22						NaN	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
23						Quarterly Daily Solar Data	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
24						NaN	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
25						NaN	NaN	NaN			NaN	
NaN	NaN	NaN	NaN	NaN	NaN					NaN	NaN	NaN
aN	NaN		NaN		NaN							
26						NaN	Month	Date	Radio Flux	10.7		
NaN	NaN	NaN	NaN	NaN	NaN				# YR	MO		
DA	HH	SPEED		DENSITY								
27						2017	8	28			82	
NaN	NaN	NaN	NaN	0.0	0.0					17	8	
28	0	285		3								
28						2017	8	29			84	
NaN	NaN	NaN	NaN	1.0	1.0					17	8	
28	1	305		3.4								
29						2017	8	30			87	
NaN	NaN	NaN	NaN	2.0	2.0					17	8	
28	2	309		1.7								

Завдання 1.3

Для кожного стовпця порахуйте, скільки в ньому заповнених клітинок (`RAW.notna().sum()`), і виведіть результат.

Очікуваний результат: 32 числа. Серед них будуть нулі та згустки великих значень.
За цими згустками видно, що в аркуші **не одна таблиця, а кілька**, вставлені поруч.

Питання: скільки таблиць в аркуші і які стовпці займає кожна?

```
In [6]: # ВАШ КОД
print(RAW.notna().sum())
```

```
0      0
1     7223
2     7202
3     7202
4     7202
5     7203
6     7203
7     7198
8     7198
9     7198
10    7198
11    7198
12    7198
13    7198
14    7198
15      0
16     32
17     26
18     26
19     26
20      0
21      0
22      0
23      0
24    601
25    601
26    608
27    602
28    602
29    602
30    602
31    602
dtype: int64
```

Відповідь 1.3:

У стовпцях з 1го по 14тий схожа кількість заповнених клітинок (У перших стовпцях на декілька рядків більше). Це наша перша таблиця.

З 16го по 19тий стовпець також схожа кількість рядків -> це друга таблиця.

З 24го по 31ий стовпець наша остання таблиця -> третя

Тобто у нас назагал є 3 таблиці.

Завдання 1.4

Над кожною таблицею є кілька текстових рядків — це службовий опис від тих, хто дані збирав. Виведіть їх і випишіть для кожної таблиці **джерело даних та одиниці вимірювання**.

Це найкорисніші п'ять хвилин роботи: жодної з цих відомостей у самій таблиці немає, і без них ви не знатимете, у чому вимірюєте.

```
In [7]: # ВАШ КОД
table_A = RAW.iloc[1:20, 1].to_string(index=False)
table_B = RAW.iloc[14:19, 16].to_string(index=False)
table_C = RAW.iloc[1:7, 26].to_string(index=False)

table = {'A': {'Origin' : RAW.iat[0,1], 'Units' : table_A},
        'B': {'Origin' : RAW.iat[13,16], 'Units' : table_B},
        'C': {'Origin' : RAW.iat[0,26], 'Units' : table_C}
}

pd.set_option('display.max_colwidth', None)
answer = pd.DataFrame(table).T

print(answer)
```

```

Origin \
A      ftp://ftp.swpc.noaa.gov/pub/lists/particle/
B  ftp://ftp.swpc.noaa.gov/pub/indices/old_indices/2017Q3_DSD.txt
C      http://umtof.umd.edu/pm/crn/

Units
A      NaN\n      :Data_list: 20
170901_Gs_part_5m.txt\n      :Created: 2017 Sep 02 0011 UTC\n# Prepare
d by the U.S. Dept. of Commerce, NOAA,...\n# Please send comments and suggestions to
SWPC....\n      # \n      # Label:
P > 1 = Particles at >1 Mev\n      # Label: P > 5 = Particles at >5 Mev\n
# Label: P >10 = Particles at >10 Mev\n      # Label: P >30 = Particles at >3
0 Mev\n      # Label: P >50 = Particles at >50 Mev\n      # Label: P>10
0 = Particles at >100 Mev\n      # Label: E>0.8 = Electrons at >0.8 Mev\n
# Label: E>2.0 = Electrons at >2.0 Mev\n      # Label: E>4.0 = Electrons at >
4.0 Mev\n      # Units: Particles = Protons/cm2-s-sr\n      # Units: Ele
ctrons = Electrons/cm2-s-sr\n      # Source: GOES-15\n
# Location: W135
B
NaN\n      Product: Daily Solar Data      quar_DSD.txt\n      :Issu
ed: 0825 UT 04 Oct 2017\n      #\n# Prep
ared by the U.S. Dept. of Commerce, NOAA...
C
NaN\n      SOHO CELIAS Proton Monitor\nSolar Wind Parameters for C
arrington Rotation (... \n      Proton speed [km/sec]\n      Pr
oton density [protons per cubic centimeter]\n
NaN
```

Відповідь 1.4:

Таблиця	Джерело	Одиниці вимірюван
A	ftp://ftp.swpc.noaa.gov/pub/lists/particle/	NaN :Data_list: 20170901_Gs_part_5m.t :Created: 2017 Sep 02 01 UTC\n# Prepared by the Dept. of Commerce, NOAA,...\n# Please send comments and suggesti to SWPC.... # # Label: P Particles at > 1 Mev # La P > 5 = Particles at > 5 M Label: P > 10 = Particles > 10 Mev# Label: P > 30 Particles at > 30 Mev# La P > 50 = Particles at > 50 # Label: P>100 = Particl > 100 Mev # Label: E>0.1 Electrons at >0.8 Mev # Label: E>2.0 = Electrons > 2.0 Mev # Label: E>4.0 Electrons at >4.0 Mev# Units: Particles = Protons/cm2-s-sr# Unit: Electrons = Electrons/cm sr # Source: GOES-15 # Location: W135
B	ftp://ftp.swpc.noaa.gov/pub/indices/old_indices/2017Q3_DSD.txt	NaN Product: Daily Sola Data quar_DSD.txt :Issue 0825 UT 04 Oct 2017 # : Prepared by the U.S. De Commerce, NOAA, Spac Weather Prediction Cen Please send comments a suggestions to SWPC.Webmaster@noa
C	http://umtof.umd.edu/pm/crn/	NaN SOHO CELIAS Prot Monitor\nSolar Wind Parameters for Carrington Rotation (hour averages Proton speed [km/sec] Proton density [protons cubic centimeter] NaN

Етап 2. Три таблиці в робочому вигляді

У файлі три таблиці. Ось що в них лежить — решту ви вже знайшли самі:

Таблиця	Стовпці аркуша	Що це	Один рядок =
A	B–O	вимірювання із супутника GOES-15: потоки частинок різної енергії	5 хвилин
B	Q–T	добовий показник сонячної активності F10.7	доба
C	Y–AF	швидкість і густина сонячного вітру	година

Завдання 2.1

Виділіть кожну таблицю в окремий `DataFrame` — назвіть їх `A`, `B` і `C`. Дайте стовпцям осмислені назви й перетворіть значення на числа (`pd.to_numeric(..., errors='coerce')` стане в пригоді).

```
In [8]: # ВАШ КОД

# A
A_table = RAW.iloc[26:7226, 1:15].to_numpy()
A_columns = ['Year', 'Month', 'Day', 'Hour:Minute', 'Julian date', 'Seconds in day']
float_columns = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']
A = pd.DataFrame(A_table, columns=A_columns)

for col in A.columns:
    if col not in float_columns:
        A[col] = pd.to_numeric(A[col], errors='coerce', downcast="integer")
    else:
        A[col] = pd.to_numeric(A[col], errors='coerce', downcast="float")

A.rename(columns={'Hour:Minute': 'Hour'}, inplace=True)
A_minutes = A['Hour'] % 100
A.insert(4, 'Minute', A_minutes)
A['Hour'] = A['Hour'] // 100

print("Таблиця A : \n", A.head(25))
print("-"*150)

# B
B_table = RAW.iloc[27:52, 16:20].to_numpy()
B_columns = ['Year', 'Month', 'Day', 'Radio Flux 10.7']
B = pd.DataFrame(B_table, columns=B_columns)

for col in B.columns:
    B[col] = pd.to_numeric(B[col], errors='coerce', downcast="integer")

print("Таблиця B : \n", B.head(25))
print("-"*150)

# C
C_table = RAW.iloc[27:628, 26:32].to_numpy()
C_columns = ['Year', 'Month', 'Day', 'Hour', 'Speed', 'Density']
C = pd.DataFrame(C_table, columns=C_columns)
```

```
for col in C.columns:
    if col != 'Density':
        C[col] = pd.to_numeric(C[col], errors='coerce', downcast="integer")
    else :
        C[col] = pd.to_numeric(C[col], errors='coerce', downcast="float")

C['Year'] = C['Year'].astype("int64") + 2000

print("Таблиця С : \n", C.head(25))
print("-"*150)
```

Таблиця А :

	Year	Month	Day	Hour	Minute	Julian date	Seconds in day	P>1	P>5	P>1
0	P>30	P>50	P>100	E>0.8	E>2.0					
0	2017	8	28	0	0	57993	0	12.00	0.260	0.180
0.1140	0.1040	0.0793	23600.0	1180.0						
1	2017	8	28	0	5	57993	300	19.40	0.690	0.314
0.1120	0.0612	0.0251	23100.0	1160.0						
2	2017	8	28	0	10	57993	600	8.48	0.168	0.130
0.0717	0.0614	0.0304	22200.0	1090.0						
3	2017	8	28	0	15	57993	900	15.40	0.566	0.198
0.0926	0.0568	0.0251	20400.0	997.0						
4	2017	8	28	0	20	57993	1200	6.41	0.156	0.118
0.0597	0.0494	0.0251	18500.0	875.0						
5	2017	8	28	0	25	57993	1500	9.94	0.495	0.310
0.2240	0.1780	0.0630	19600.0	950.0						
6	2017	8	28	0	30	57993	1800	6.47	0.344	0.306
0.0650	0.0547	0.0268	19800.0	952.0						
7	2017	8	28	0	35	57993	2100	11.00	0.334	0.121
0.0597	0.0494	0.0251	20100.0	966.0						
8	2017	8	28	0	40	57993	2400	6.36	0.210	0.171
0.1130	0.1030	0.0572	20600.0	993.0						
9	2017	8	28	0	45	57993	2700	6.41	0.188	0.150
0.0916	0.0814	0.0570	21600.0	1030.0						
10	2017	8	28	0	50	57993	3000	6.34	0.180	0.142
0.0837	0.0734	0.0491	21400.0	1010.0						
11	2017	8	28	0	55	57993	3300	9.13	0.308	0.175
0.0992	0.0583	0.0251	20400.0	936.0						
12	2017	8	28	1	0	57993	3600	13.60	0.379	0.121
0.0597	0.0494	0.0251	20200.0	907.0						
13	2017	8	28	1	5	57993	3900	11.10	0.285	0.122
0.0597	0.0494	0.0251	20600.0	896.0						
14	2017	8	28	1	10	57993	4200	6.24	0.212	0.174
0.0795	0.0539	0.0251	19900.0	866.0						
15	2017	8	28	1	15	57993	4500	11.30	0.459	0.193
0.0919	0.0699	0.0422	19600.0	803.0						
16	2017	8	28	1	20	57993	4800	10.40	0.512	0.259
0.0597	0.0494	0.0251	19200.0	798.0						
17	2017	8	28	1	25	57993	5100	5.18	0.187	0.148
0.0900	0.0798	0.0348	18900.0	766.0						
18	2017	8	28	1	30	57993	5400	7.09	0.275	0.157
0.0777	0.0673	0.0308	18700.0	736.0						
19	2017	8	28	1	35	57993	5700	5.38	0.179	0.140
0.0823	0.0716	0.0471	18000.0	685.0						
20	2017	8	28	1	40	57993	6000	8.68	0.324	0.135
0.0730	0.0524	0.0251	16900.0	622.0						
21	2017	8	28	1	45	57993	6300	4.64	0.188	0.150
0.0597	0.0494	0.0251	15400.0	529.0						
22	2017	8	28	1	50	57993	6600	4.82	0.168	0.130
0.0714	0.0610	0.0288	14800.0	485.0						
23	2017	8	28	1	55	57993	6900	7.50	0.332	0.150
0.0664	0.0509	0.0251	14900.0	463.0						
24	2017	8	28	2	0	57993	7200	5.26	0.359	0.321
0.0743	0.0640	0.0397	15000.0	475.0						

Таблиця В :

	Year	Month	Day	Radio Flux 10.7
0	2017	8	28	82
1	2017	8	29	84
2	2017	8	30	87
3	2017	8	31	92
4	2017	9	1	93
5	2017	9	2	100
6	2017	9	3	120
7	2017	9	4	140
8	2017	9	5	121
9	2017	9	6	133
10	2017	9	7	129
11	2017	9	8	117
12	2017	9	9	107
13	2017	9	10	100
14	2017	9	11	80
15	2017	9	12	76
16	2017	9	13	75
17	2017	9	14	74
18	2017	9	15	73
19	2017	9	16	72
20	2017	9	17	72
21	2017	9	18	72
22	2017	9	19	71
23	2017	9	20	74
24	2017	9	21	73

Таблиця С :

	Year	Month	Day	Hour	Speed	Density
0	2017	8	28	0	285	3.0
1	2017	8	28	1	305	3.4
2	2017	8	28	2	309	1.7
3	2017	8	28	3	304	2.0
4	2017	8	28	4	302	2.3
5	2017	8	28	5	304	2.4
6	2017	8	28	6	319	3.2
7	2017	8	28	7	319	2.8
8	2017	8	28	8	319	3.2
9	2017	8	28	9	306	4.0
10	2017	8	28	10	305	4.0
11	2017	8	28	11	307	3.3
12	2017	8	28	12	303	2.9
13	2017	8	28	13	306	2.8
14	2017	8	28	14	294	3.2
15	2017	8	28	15	303	2.0
16	2017	8	28	16	307	1.5
17	2017	8	28	17	309	0.9
18	2017	8	28	18	302	1.5
19	2017	8	28	19	294	2.0
20	2017	8	28	20	297	3.2
21	2017	8	28	21	299	2.8
22	2017	8	28	22	309	2.9
23	2017	8	28	23	309	3.2
24	2017	8	29	0	325	2.1

Завдання 2.2

У кожній таблиці дата й час записані **по-різному** й розкидані по кількох стовпцях. Зберіть для кожної таблиці один стовпець `ts` типу `datetime`.

Після цього обов'язково виведіть для кожної таблиці найранішу й найпізнішу дату.

Очікуваний результат: усі три таблиці мають лежати в межах серпня–вересня 2017 року. Якщо у вас вийшов інший рік — придивіться, як записано рік у тій таблиці, і виправте.

```
In [9]: # ВАШ КОД

tableA_time = pd.to_datetime(A[['Year', 'Month', 'Day', 'Hour', 'Minute']], errors=
A.insert(A.shape[1], 'ts', tableA_time.sort_values(ascending=True))

table_B_time = pd.to_datetime(B[['Year', 'Month', 'Day']], errors='coerce', format=
B.insert(B.shape[1], 'ts', table_B_time.sort_values(ascending=True))

table_C_time = pd.to_datetime(C[['Year', 'Month', 'Day', 'Hour']], errors='coerce',
C.insert(C.shape[1], 'ts', table_C_time.sort_values(ascending=True)))
```

Самоперевірка етапу 2. Клітинка має виконатися без помилок.

```
In [10]: assert len(A) == 7200, 'у таблиці A має бути 7200 рядків, а є %d' % len(A)
assert len(B) == 25, 'у таблиці B має бути 25 рядків, а є %d' % len(B)
assert len(C) == 601, 'у таблиці C має бути 601 рядок, а є %d' % len(C)
for назва, т in [('A', A), ('B', B), ('C', C)]:
    рік = pd.to_datetime(т['ts']).dt.year
    assert рік.min() == 2017 and рік.max() == 2017, 'у таблиці %s рік не 2017 – пер
print('Етап 2 пройдено')
```

Етап 2 пройдено

Етап 3. Паспорт даних

Завдання 3.1

Для кожної таблиці виведіть: розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків і описові статистики (`describe()`).

```
In [11]: # ВАШ КОД

print("Таблиця A : \n")
print("Розмір: ", A.shape[0], "x", A.shape[1])
print("Стовпці: ", A.columns.tolist())
print("Типи даних: \n---\n", A.dtypes, "\n---")
```

```
print("Кількість пропусків", A.isnull().sum().tolist())
print("Кількість дуплікатів", A.duplicated().sum())
print("Описові статистики: \n", A.describe(include="all"))
print("-"*170)
print("Таблиця B : \n")
print("Розмір: ", B.shape[0], "x", B.shape[1])
print("Стовпці: ", B.columns.tolist())
print("Типи даних: \n---\n", B.dtypes, "\n---")
print("Кількість пропусків", B.isnull().sum().tolist())
print("Кількість дуплікатів", B.duplicated().sum())
print("Описові статистики: \n", B.describe(include="all"))
print("-"*170)
print("Таблиця C : \n")
print("Розмір: ", C.shape[0], "x", C.shape[1])
print("Стовпці: ", C.columns.tolist())
print("Типи даних: \n---\n", C.dtypes, "\n---")
print("Кількість пропусків", C.isnull().sum().tolist())
print("Кількість дуплікатів", C.duplicated().sum())
print("Описові статистики: \n", C.describe(include="all"))
```

Таблиця А :

Розмір: 7200 x 16

Стовпці: ['Year', 'Month', 'Day', 'Hour', 'Minute', 'Julian date', 'Seconds in day', 'P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0', 'ts']

Типи даних:

```

---
Year                int16
Month               int8
Day                int8
Hour               int16
Minute             int16
Julian date        int32
Seconds in day     int32
P>1                float32
P>5                float32
P>10               float32
P>30               float32
P>50               float32
P>100              float32
E>0.8              float32
E>2.0              float32
ts                 datetime64[us]
dtype: object
---

```

Кількість пропусків [0, 0, 0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 0]

Кількість дуплікатів 0

Описові статистики:

	Year	Month	Day	Hour	Minute	Julian date	Se
conds in day	P>1	P>5	P>10	P>30	P>50		
P>100	E>0.8	\					
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000000	
	7200.000000	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000
mean	2017.0	8.840000	13.960000	11.500000	27.500000	58005.000000	4
	3050.000000	689.325073	156.626053	76.474014	18.838604	7.446839	1.65929
min	2017.0	8.000000	1.000000	0.000000	0.000000	57993.000000	
	0.000000	0.429000	0.151000	0.117000	0.057600	0.045500	0.02240
25%	2017.0	9.000000	7.000000	5.750000	13.750000	57999.000000	2
	1525.000000	11.700000	0.279000	0.186000	0.098400	0.059700	0.02630
50%	2017.0	9.000000	13.000000	11.500000	27.500000	58005.000000	4
	3050.000000	55.400002	0.632000	0.376000	0.146000	0.086600	0.03630
75%	2017.0	9.000000	19.000000	17.250000	41.250000	58011.000000	6
	4575.000000	560.000000	107.000000	35.799999	0.990000	0.249000	0.06700
max	2017.0	9.000000	31.000000	23.000000	55.000000	58017.000000	8
	6100.000000	32500.000000	2890.000000	1330.000000	399.000000	191.000000	61.50000
std	0.0	0.366632	8.775483	6.922667	17.261461	7.211603	2
	4943.113498	1625.968506	390.921875	222.362625	67.265594	28.620523	7.15889

```

      E>2.0      ts
count    7197.000000    7200
mean    11401.502930    2017-09-09 11:57:30
min       0.133000    2017-08-28 00:00:00
25%      552.000000    2017-09-03 05:58:45
50%      4290.000000    2017-09-09 11:57:30
75%     14100.000000    2017-09-15 17:56:15
max     110000.000000    2017-09-21 23:55:00
std      16920.617188    NaN

```

--

Таблиця В :

Розмір: 25 x 5

Стовпці: ['Year', 'Month', 'Day', 'Radio Flux 10.7', 'ts']

Типи даних:

```

---
Year                                int16
Month                              int8
Day                                int8
Radio Flux 10.7                    int16
ts                                  datetime64[us]
dtype: object

```

Кількість пропусків [0, 0, 0, 0, 0]

Кількість дуплікатів 0

Описові статистики:

```

      Year      Month      Day  Radio Flux 10.7      ts
count    25.0    25.000000    25.000000    25.000000    25
mean    2017.0     8.840000    13.960000    92.680000    2017-09-09 00:00:00
min     2017.0     8.000000     1.000000    71.000000    2017-08-28 00:00:00
25%     2017.0     9.000000     7.000000    74.000000    2017-09-03 00:00:00
50%     2017.0     9.000000    13.000000    84.000000    2017-09-09 00:00:00
75%     2017.0     9.000000    19.000000   107.000000    2017-09-15 00:00:00
max     2017.0     9.000000    31.000000   140.000000    2017-09-21 00:00:00
std        0.0     0.374166     8.955817    22.206831    NaN

```

--

Таблиця С :

Розмір: 601 x 7

Стовпці: ['Year', 'Month', 'Day', 'Hour', 'Speed', 'Density', 'ts']

Типи даних:

```

---
Year                                int64
Month                              int8
Day                                int8
Hour                               int8
Speed                             int16
Density                          float32
ts                                  datetime64[us]
dtype: object

```

Кількість пропусків [0, 0, 0, 0, 0, 0, 0]

Кількість дуплікатів 0

Описові статистики:

	Year	Month	Day	Hour	Speed	Density	
ts							
count	601.0	601.000000	601.000000	601.000000	601.000000	601.000000	
601							
mean	2017.0	8.840266	13.973378	11.480865	506.547421	2.907654	2017-09-09 12:00:00
min	2017.0	8.000000	1.000000	0.000000	-1.000000	-1.000000	2017-08-28 00:00:00
25%	2017.0	9.000000	7.000000	5.000000	447.000000	1.800000	2017-09-03 06:00:00
50%	2017.0	9.000000	13.000000	11.000000	517.000000	2.400000	2017-09-09 12:00:00
75%	2017.0	9.000000	19.000000	17.000000	584.000000	3.400000	2017-09-15 18:00:00
max	2017.0	9.000000	31.000000	23.000000	806.000000	31.700001	2017-09-22 00:00:00
std	0.0	0.366664	8.781000	6.938063	125.073291	2.637873	
NaN							

Завдання 3.2 — дивні значення

Подивіться на рядок `min` у таблиці `describe()` для таблиці `C`.

Швидкість і густина — фізичні величини, які **не можуть бути від'ємними**. Знайдіть від'ємні значення, порахуйте, скільки їх, і поверніться до службового опису з завдання 1.4: там написано, що вони означають.

Замініть ці значення на `NaN` і покажіть, як після цього змінилися середнє й мінімум.

Очікуваний результат: два числа «до» і «після» для кожного зі стовпців.

```
In [12]: # ВАШ КОД
count_speed = (C["Speed"] < 0).sum()
count_density = (C["Density"] < 0).sum()
print(f"Від'ємних у Speed: {count_speed}")
print(f"Від'ємних у Density: {count_density}")

print("\nДо зміни : \n", C[["Speed", "Density"]].describe())

C.loc[C["Speed"] < 0, "Speed"] = np.nan
C.loc[C["Density"] < 0, "Density"] = np.nan
print("\n", "-"*50, "\n")
print("Після зміни : \n", C[["Speed", "Density"]].describe())
```

Від'ємних у Speed: 8
Від'ємних у Density: 8

До зміни :

	Speed	Density
count	601.000000	601.000000
mean	506.547421	2.907654
std	125.073291	2.637873
min	-1.000000	-1.000000
25%	447.000000	1.800000
50%	517.000000	2.400000
75%	584.000000	3.400000
max	806.000000	31.700001

Після зміни :

	Speed	Density
count	593.000000	593.000000
mean	513.394604	2.960371
std	111.025317	2.615965
min	277.000000	0.100000
25%	449.000000	1.800000
50%	520.000000	2.500000
75%	585.000000	3.400000
max	806.000000	31.700001

Відповідь 3.2: це пропущені значення, які були втрачені або не виміряні. Їх разом 16 елементів, по 8 у кожній колонці.

Завдання 3.3 — питання на подумати

У таблиці A є стовпці, для яких `describe()` слухняно рахує середнє й стандартне відхилення, хоча ці числа не мають жодного сенсу.

Знайдіть їх і поясніть своїми словами, чому середнє для них безглузде.

Підказка: не кожне число в таблиці є результатом вимірювання.

Це питання не має однієї «правильної» відповіді з підручника — на нього ви відповідаєте з власного здорового глузду. Формальне пояснення буде на лекції 3.

```
In [13]: # ВАШ КОД
print("Описові статистики таблиці A: \n", A.describe(include="all"))
```

Описові статистики таблиці A:

	Year	Month	Day	Hour	Minute	Julian date	Se
conds in day	P>1	P>5	P>10	P>30	P>50		
P>100	E>0.8	\					
count	7200.0	7200.000000	7200.000000	7200.000000	7200.000000	7200.000000	
7200.000000	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000
mean	2017.0	8.840000	13.960000	11.500000	27.500000	58005.000000	4
3050.000000	689.325073	156.626053	76.474014	18.838604	7.446839		1.
65929	81862.742188						
min	2017.0	8.000000	1.000000	0.000000	0.000000	57993.000000	
0.000000	0.429000	0.151000	0.117000	0.057600	0.045500	0.022	
40	4.300000						
25%	2017.0	9.000000	7.000000	5.750000	13.750000	57999.000000	2
1525.000000	11.700000	0.279000	0.186000	0.098400	0.059700		0.
02630	22200.000000						
50%	2017.0	9.000000	13.000000	11.500000	27.500000	58005.000000	4
3050.000000	55.400002	0.632000	0.376000	0.146000	0.086600		0.
03630	68600.000000						
75%	2017.0	9.000000	19.000000	17.250000	41.250000	58011.000000	6
4575.000000	560.000000	107.000000	35.799999	0.990000	0.249000		0.
06700	143000.000000						
max	2017.0	9.000000	31.000000	23.000000	55.000000	58017.000000	8
6100.000000	32500.000000	2890.000000	1330.000000	399.000000	191.000000		61.
50000	259000.000000						
std	0.0	0.366632	8.775483	6.922667	17.261461	7.211603	2
4943.113498	1625.968506	390.921875	222.362625	67.265594	28.620523		7.
15889	65728.000000						
	E>2.0		ts				
count	7197.000000		7200				
mean	11401.502930	2017-09-09	11:57:30				
min	0.133000	2017-08-28	00:00:00				
25%	552.000000	2017-09-03	05:58:45				
50%	4290.000000	2017-09-09	11:57:30				
75%	14100.000000	2017-09-15	17:56:15				
max	110000.000000	2017-09-21	23:55:00				
std	16920.617188		NaN				

Відповідь 3.3: Середнє для будь-якої частини дати безглузде - День, година, хвилина та секунди повторюються з періодичністю, рік один і той самий, а місяць усереднювати потреби немає

Етап 4. Об'єднання трьох таблиць в одну

Таблиці мають різний крок у часі: 5 хвилин, година і доба. Щоб працювати з ними разом, потрібна спільна сітка. Беремо **годину**.

Кожна таблиця потребує своєї дії:

1. **A: 5 хвилин** → **година**. Кілька рядків згортаються в один — це `resample` + агрегація.
2. **C: уже година**. Просте з'єднання по спільному стовпцю `ts` — `merge`.
3. **B: доба** → **година**. Навпаки, одне значення розтягується на 24 години доби.

Завдання 4.1

Зведіть таблицю A до погодинної сітки. Для кожного потоку порахуйте **два** числа за годину: середнє і максимум.

```
In [14]: # ВАШ КОД
coll = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']
A_hourly = (
    A.set_index('ts')[coll].resample('1h').agg(['mean', 'max'])
)
A_hourly.columns = ['_'.join(col).strip('_') for col in A_hourly.columns]
A_hourly = A_hourly.reset_index()
A_hourly['Year'] = A_hourly['ts'].dt.year
A_hourly['Month'] = A_hourly['ts'].dt.month
A_hourly['Day'] = A_hourly['ts'].dt.day
A_hourly['Hour'] = A_hourly['ts'].dt.hour
print(A_hourly.head(25))
```

		ts	P>1_mean	P>1_max	P>5_mean	P>5_max	P>10_mean	P>10_max	P
>30_mean	P>30_max	P>50_mean	P>50_max	P>100_mean	P>100_max	E>0.8_mean	E>0.8_max		
max	E>2.0_mean	E>2.0_max	\						
0	2017-08-28 00:00:00	9.778333	19.40	0.324917	0.690	0.192917	0.314		
0.098850	0.2240	0.077583	0.1780	0.040692	0.0793	20941.666016	2360		
0.0	1011.583313	1180.000000							
1	2017-08-28 01:00:00	7.994167	13.60	0.291667	0.512	0.156583	0.259		
0.072583	0.0919	0.058700	0.0798	0.029950	0.0471	18091.666016	2060		
0.0	713.000000	907.000000							
2	2017-08-28 02:00:00	6.099167	10.80	0.283000	0.479	0.177667	0.321		
0.084450	0.1210	0.069642	0.1110	0.035025	0.0595	14108.333008	1500		
0.0	380.833344	475.000000							
3	2017-08-28 03:00:00	4.752500	7.76	0.236833	0.329	0.166333	0.244		
0.087658	0.1800	0.068092	0.1390	0.035517	0.0818	12766.666992	1310		
0.0	267.916656	284.000000							
4	2017-08-28 04:00:00	4.158333	6.62	0.334917	0.539	0.208333	0.309		
0.084800	0.1530	0.072850	0.1420	0.039708	0.0694	11333.333008	1230		
0.0	203.000000	234.000000							
5	2017-08-28 05:00:00	2.904166	4.62	0.240333	0.410	0.166833	0.284		
0.077292	0.1140	0.063692	0.0932	0.034883	0.0669	9685.833008	1070		
0.0	152.416672	180.000000							
6	2017-08-28 06:00:00	3.421667	4.94	0.281917	0.430	0.163417	0.249		
0.083842	0.1160	0.066992	0.1060	0.037583	0.0818	8650.000000	920		
0.0	127.916664	137.000000							
7	2017-08-28 07:00:00	2.978333	3.93	0.324583	0.601	0.196667	0.316		
0.100283	0.1650	0.076817	0.1550	0.039333	0.0769	6424.166504	760		
0.0	95.491669	110.000000							
8	2017-08-28 08:00:00	2.611667	3.96	0.302333	0.495	0.187750	0.280		
0.095350	0.2190	0.074467	0.1380	0.037683	0.0620	4602.500000	521		
0.0	70.016670	79.800003							
9	2017-08-28 09:00:00	2.554167	3.98	0.293667	0.608	0.179583	0.258		
0.096558	0.1360	0.076192	0.1090	0.038833	0.0595	4761.666504	496		
0.0	82.341667	89.599998							
10	2017-08-28 10:00:00	2.387500	3.32	0.299083	0.491	0.192833	0.407		
0.097425	0.1410	0.067433	0.1060	0.038392	0.0744	4598.333496	495		
0.0	89.700005	108.000000							
11	2017-08-28 11:00:00	2.383333	3.16	0.287583	0.459	0.172917	0.326		
0.083492	0.1410	0.063942	0.1020	0.031692	0.0645	5865.833496	724		
0.0	124.833336	156.000000							
12	2017-08-28 12:00:00	2.512500	3.92	0.253750	0.418	0.167750	0.257		
0.092883	0.1280	0.070075	0.0962	0.040433	0.0719	9123.333008	1100		
0.0	208.416672	246.000000							
13	2017-08-28 13:00:00	3.254167	4.92	0.306083	0.671	0.175250	0.358		
0.085567	0.1500	0.062367	0.0892	0.030258	0.0491	12200.000000	1300		
0.0	264.333344	271.000000							
14	2017-08-28 14:00:00	2.590000	3.28	0.244500	0.373	0.193250	0.334		
0.113792	0.2740	0.090042	0.1840	0.048933	0.0823	13875.000000	1430		
0.0	303.166656	323.000000							
15	2017-08-28 15:00:00	3.093333	4.32	0.292917	0.431	0.198000	0.338		
0.089842	0.1530	0.067958	0.1430	0.035875	0.0620	15925.000000	1700		
0.0	406.250000	478.000000							
16	2017-08-28 16:00:00	3.616667	5.11	0.306500	0.524	0.200750	0.363		
0.100758	0.1610	0.077200	0.1250	0.044525	0.0769	18016.666016	1880		
0.0	557.000000	599.000000							
17	2017-08-28 17:00:00	3.972500	5.23	0.298500	0.656	0.193917	0.359		
0.102992	0.1950	0.082908	0.1450	0.043333	0.0854	20050.000000	2080		

```

0.0 591.583313 615.000000
18 2017-08-28 18:00:00 4.538333 5.85 0.275083 0.383 0.163750 0.252
0.090375 0.1920 0.072483 0.1610 0.035600 0.0620 20883.333984 2190
0.0 592.416687 609.000000
19 2017-08-28 19:00:00 4.429167 4.94 0.220667 0.302 0.170167 0.256
0.091525 0.1310 0.071167 0.1210 0.035892 0.0509 22058.333984 2220
0.0 585.333313 605.000000
20 2017-08-28 20:00:00 5.457500 7.04 0.202333 0.280 0.142083 0.190
0.072700 0.1060 0.053517 0.0683 0.026742 0.0360 23108.333984 2410
0.0 666.833313 727.000000
21 2017-08-28 21:00:00 5.880000 8.62 0.226750 0.411 0.168833 0.226
0.097450 0.1440 0.081417 0.1330 0.047583 0.0707 24550.000000 2480
0.0 773.083313 792.000000
22 2017-08-28 22:00:00 6.835833 8.59 0.237583 0.521 0.181167 0.409
0.102192 0.2120 0.070900 0.1260 0.036350 0.0779 25641.666016 2680
0.0 775.166687 804.000000
23 2017-08-28 23:00:00 6.745000 9.76 0.274250 0.392 0.198000 0.282
0.112458 0.1470 0.083500 0.1210 0.042383 0.0856 24866.666016 2600
0.0 742.166687 778.000000
24 2017-08-29 00:00:00 5.596667 7.13 0.235500 0.392 0.174583 0.274
0.096425 0.1650 0.079367 0.1540 0.043725 0.0879 20658.333984 2330
0.0 494.666656 699.000000

```

	Year	Month	Day	Hour
0	2017	8	28	0
1	2017	8	28	1
2	2017	8	28	2
3	2017	8	28	3
4	2017	8	28	4
5	2017	8	28	5
6	2017	8	28	6
7	2017	8	28	7
8	2017	8	28	8
9	2017	8	28	9
10	2017	8	28	10
11	2017	8	28	11
12	2017	8	28	12
13	2017	8	28	13
14	2017	8	28	14
15	2017	8	28	15
16	2017	8	28	16
17	2017	8	28	17
18	2017	8	28	18
19	2017	8	28	19
20	2017	8	28	20
21	2017	8	28	21
22	2017	8	28	22
23	2017	8	28	23
24	2017	8	29	0

Завдання 4.2

У вашому завданні названо агрегат (середнє , максимум , медіана або мінімум).

Візьміть ваш потік за одну добу, порахуйте для нього по годинах ваш агрегат і ще один на ваш вибір, і побудуйте обидві криві на одному графіку.

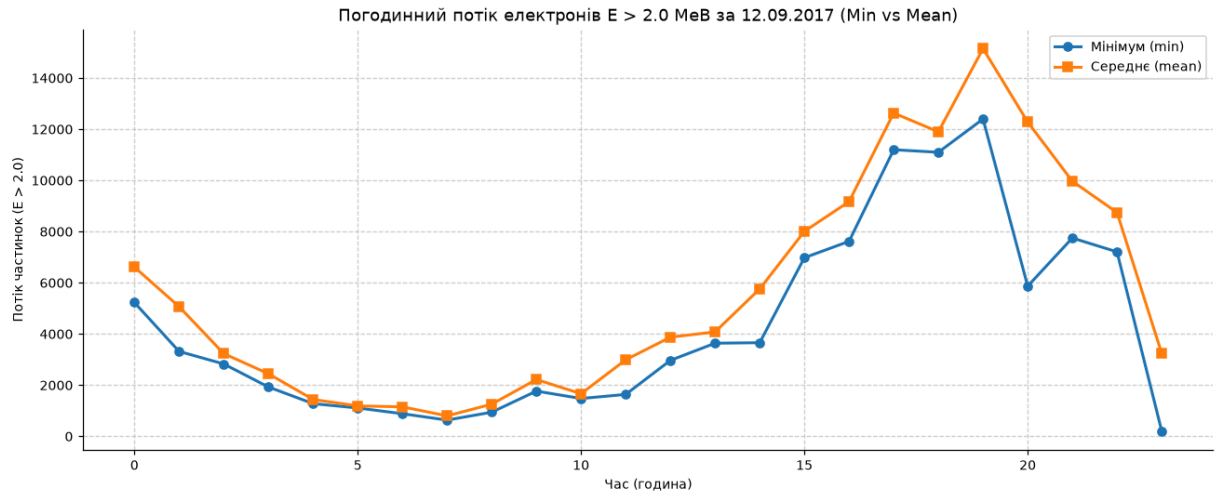
Питання: чим вони відрізняються і в якій ситуації різниця між ними стане суттєвою?

```
In [15]: # ВАШ КОД
My_day_mean = A_hourly[
    (A_hourly['ts'].dt.year == 2017) & (A_hourly['ts'].dt.month == 9) & (A_hourly['
'].copy()[['E>2.0_mean', 'ts']])

My_day_A = A[
    (A['ts'].dt.year == 2017) & (A['ts'].dt.month == 9) & (A['ts'].dt.day == 12)
].copy()[['E>2.0', 'ts']]
My_day_min = My_day_A.set_index('ts')['E>2.0'].resample('1h').min()

# 4. Побудова двох кривих на одному графіку
plt.figure(figsize=(12, 5))
plt.plot(
    My_day_min.index.hour,
    My_day_min,
    label='Мінімум (min)',
    marker='o',
    linewidth=2,
    color='tab:blue',
)
plt.plot(
    My_day_mean['ts'].dt.hour,
    My_day_mean['E>2.0_mean'],
    label='Середнє (mean)',
    marker='s',
    linewidth=2,
    color='tab:orange',
)

plt.title(
    'Погодинний потік електронів E > 2.0 МеВ за 12.09.2017 (Min vs Mean)',
    fontsize=12,
)
plt.xlabel('Час (година)')
plt.ylabel('Потік частинок (E > 2.0)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
```



Відповідь 4.2:

Крива середнього значення завжди вища за криву мінімального значення

Існують моменти, коли криві майже зливаються (~2-10), тобто потік був майже рівномірним, максимальне від мінімального далеко не відходило

Однак є також протилежні хм моменти - великі розриви (20-та година), які показують, що в той момент були висока динаміка та нестабільність

Різниця між даними важлива, коли : потрібно знати загальну оцінку потоку частинок (середнє), або ж коли потрібно знати, за яку межу потік не опускався

Завдання 4.3

Об'єднайте три таблиці в одну — назвіть її **M** . Виведіть її розмір і всі рядки, у яких є хоча б один пропуск.

Зверніть увагу: таблиці закінчуються не в один і той самий момент. Тому результат залежить від того, який тип об'єднання ви оберете (`how='inner'` чи `how='outer'`). Спробуйте обидва, подивіться, чим відрізняється кількість рядків, і поясніть свій вибір.

```
In [16]: # ВАШ КОД
hourly_keys = ['Year', 'Month', 'Day', 'Hour', 'ts']
daily_keys = ['Year', 'Month', 'Day', 'ts']

M_outer = pd.merge(A_hourly, C, on=hourly_keys, how='outer')
M_outer = pd.merge(M_outer, B, on=daily_keys, how='outer')

M_inner = pd.merge(A_hourly, C, on=hourly_keys, how='inner')
M_inner = pd.merge(M_inner, B, on=daily_keys, how='inner')

print('Розмір M (outer):', M_outer.shape)
print('Розмір M (inner):', M_inner.shape)

null_rows_out = M_outer[M_outer.isna().any(axis=1)]
```

```
null_rows_inn = M_inner[M_inner.isna().any(axis=1)]

print('\nКількість рядків із пропусками в M_outer:', len(null_rows_out))
print('Рядки з пропусками:')
display(null_rows_out)

print('\nКількість рядків із пропусками в M_inner:', len(null_rows_inn))
print('Рядки з пропусками:')
display(null_rows_inn)

M = M_outer.copy()
```

Розмір M (outer): (601, 24)

Розмір M (inner): (25, 24)

Кількість рядків із пропусками в M_outer: 576

Рядки з пропусками:

	ts	P>1_mean	P>1_max	P>5_mean	P>5_max	P>10_mean	P>10_max	P>30_mean
1	2017-08-28 01:00:00	7.994167	13.600000	0.291667	0.512	0.156583	0.259	0.0729
2	2017-08-28 02:00:00	6.099167	10.800000	0.283000	0.479	0.177667	0.321	0.0844
3	2017-08-28 03:00:00	4.752500	7.760000	0.236833	0.329	0.166333	0.244	0.0870
4	2017-08-28 04:00:00	4.158333	6.620000	0.334917	0.539	0.208333	0.309	0.0844
5	2017-08-28 05:00:00	2.904166	4.620000	0.240333	0.410	0.166833	0.284	0.0770
...	...	...	...	...	...	...	...	...
596	2017-09-21 20:00:00	42.102501	78.900002	0.397500	0.833	0.226833	0.429	0.0890
597	2017-09-21 21:00:00	50.783337	71.699997	0.489250	0.788	0.317583	0.501	0.1210
598	2017-09-21 22:00:00	39.000000	48.799999	0.350000	0.536	0.200250	0.310	0.0940
599	2017-09-21 23:00:00	28.824999	42.099998	0.465000	0.861	0.257833	0.506	0.1130
600	2017-09-22 00:00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN

576 rows × 24 columns

◀

▶

Кількість рядків із пропусками в M_inner: 0

Рядки з пропусками:

ts	P>1_mean	P>1_max	P>5_mean	P>5_max	P>10_mean	P>10_max	P>30_mean	P>30_max
----	----------	---------	----------	---------	-----------	----------	-----------	----------

◀

▶

Відповідь 4.3: Я обрала залишити таблицю з зовнішнім з'єднанням. Саме у такій таблиці жодні дані не пропали, хоча вона і займає багато місця

Завдання 4.4 — поділ на групи

Щоб далі було що з чим порівнювати, поділіть години на три групи за значенням **максимуму потоку $P > 10$ за годину**:

Група	Умова
тиха	менше 10
активна	від 10 до 1000
дуже активна	1000 і більше

Додайте до `M` стовпець `група` і виведіть, скільки годин потрапило в кожную.

Це не вигадані числа: саме такі пороги використовує метеослужба космічної погоди NOAA. Чому вони саме такі — питання не сьогоднішньої роботи.

```
In [17]: # ВАШ КОД
bins = [-np.inf, 10, 1000, np.inf]
labels = ['тиха', 'активна', 'дуже активна']

M['група'] = pd.cut(M['P>10_max'], bins=bins, labels=labels, right=False)

print('Кількість годин у кожній групі:')
print(M['група'].value_counts())
```

Кількість годин у кожній групі:

```
група
тиха          411
активна       171
дуже активна   18
Name: count, dtype: int64
```

Самоперевірка етапу 4.

```
In [18]: assert M is not None, 'таблиця M не побудована'
assert 'ts' in M.columns, 'у таблиці M потрібен стовпець ts'
assert 'група' in M.columns, 'у таблиці M потрібен стовпець «група»'
assert 590 <= len(M) <= 605, 'очікується близько 600 рядків, а є %d – перевірте об'єкт'
print('Етап 4 пройдено:', len(M), 'рядків')
```

Етап 4 пройдено: 601 рядків

Етап 5. П'ять графіків — п'ять питань

Графік будується як **відповідь на питання**, а не «щоб був».

Вимоги до кожного графіка:

- підписані обидві осі, з одиницями вимірювання;
- заголовок;
- під графіком, у текстовій клітинці, — **одне речення про дані**. Не «на графіку зображено потік протонів», а твердження на кшталт «більшість годин потік не перевищує 60, але кілька годин дають значення у сотні разів більші».

Не кожне питання має відповідь «так, зв'язок є». Якщо на графіку нічого не видно — так і напишіть. Відсутність зв'язку — це теж результат, і чесно про неї написати важливіше, ніж підбирати ракурс, у якому «щось наче проглядається».

Графік 1. Як розподілені значення вашого потоку?

Побудуйте гістограму вашого потоку (середнє за годину). Порахуйте окремо середнє й медіану цього стовпця.

Якщо гістограма вийде нечитабельною — стовпчик біля нуля і порожньо далі — не залишайте так. Спробуйте побудувати гістограму від логарифма значень (`np.log10`) і подивіться, що зміниться.

Питання: чому середнє й медіана так сильно відрізняються?

```
In [19]: # ВАШ КОД
data = M['E>2.0_mean'].dropna()

mean_val = data.mean()
median_val = data.median()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(data, bins=40, color='skyblue', edgecolor='black')
axes[0].axvline(
    mean_val,
    color='red',
    linestyle='--',
    linewidth=2,
    label=f'Середнє ({mean_val:.1f})',
)
axes[0].axvline(
    median_val,
    color='green',
    linestyle='-',
    linewidth=2,
    label=f'Медіана ({median_val:.1f})',
)
```

```

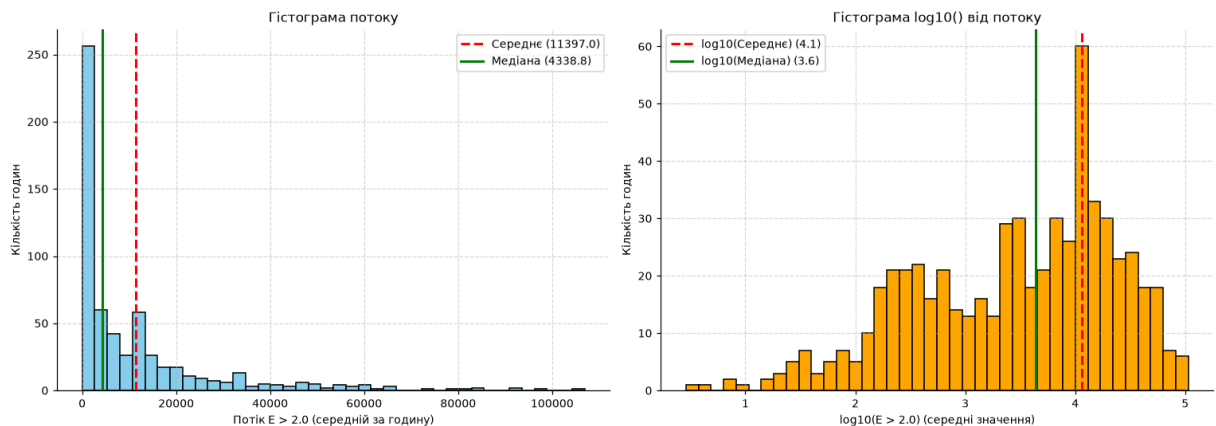
axes[0].set_title('Гістограма потоку')
axes[0].set_xlabel('Потік E > 2.0 (середній за годину)')
axes[0].set_ylabel('Кількість годин')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.5)

log_data = np.log10(data[data > 0])

axes[1].hist(log_data, bins=40, color='orange', edgecolor='black')
axes[1].axvline(
    np.log10(mean_val),
    color='red',
    linestyle='--',
    linewidth=2,
    label=f'log10(Середнє) ({np.log10(mean_val):.1f})',
)
axes[1].axvline(
    np.log10(median_val),
    color='green',
    linestyle='-',
    linewidth=2,
    label=f'log10(Медіана) ({np.log10(median_val):.1f})',
)
axes[1].set_title('Гістограма log10() від потоку')
axes[1].set_xlabel('log10(E > 2.0) (середні значення)')
axes[1].set_ylabel('Кількість годин')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Висновок: Середнє та медіана відрізняються через наявність рідкісних, однак екстримально великих значень. Середнє дуже чутливе до таких сплесків, тому зміщене правіше від медіани

Графік 2. Чи відрізняється швидкість вітру між групами годин?

Побудуйте діаграму розмаху (`boxplot`) швидкості сонячного вітру окремо для кожної групи з завдання 4.4.

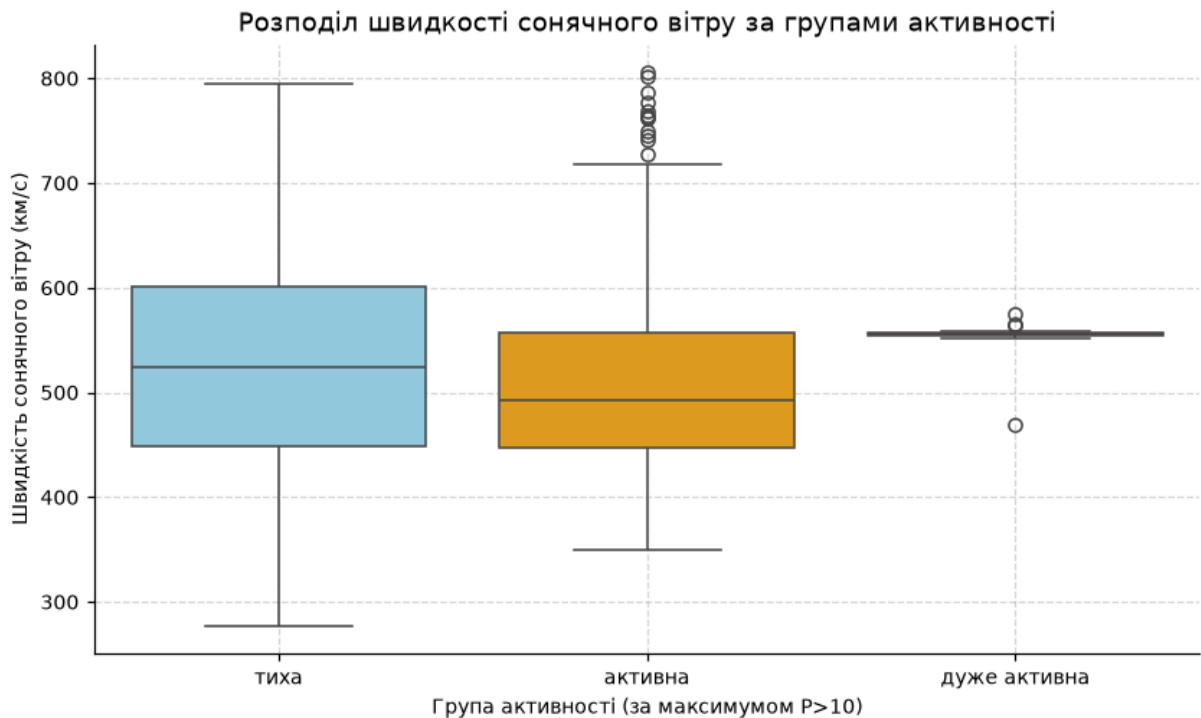
```
In [20]: # ВАШ КОД
plt.figure(figsize=(9, 5))
sns.boxplot(
    data=M,
    x='група',
    y='Speed',
    order=['тиха', 'активна', 'дуже активна'],
    palette=['skyblue', 'orange', 'crimson'],
)

plt.title(
    'Розподіл швидкості сонячного вітру за групами активності', fontsize=12
)
plt.xlabel('Група активності (за максимумом P>10)')
plt.ylabel('Швидкість сонячного вітру (км/с)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

C:\Users\Sofiya\AppData\Local\Temp\ipykernel_21204\2202295845.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(
```



Висновок: Зі збільшенням активності сонячних протонів медіанна швидкість сонячного вітру помітно від тихої групи до дуже активної. Хоч медіана на активній групі зменшилась, з'явилися викиди поза 800 км/с

Графік 3. Чи пов'язані змінні вашої пари?

Побудуйте діаграму розсіювання для пари з вашого завдання. Порахуйте коефіцієнт кореляції між ними.

```
In [21]: # ВАШ КОД
corr_pearson = M['Speed'].corr(M['Density'])
corr_spearman = M['Speed'].corr(M['Density'], method='spearman')

print(f'Коефіцієнт кореляції Пірсона: {corr_pearson:.3f}')
print(f'Коефіцієнт кореляції Спірмена: {corr_spearman:.3f}')

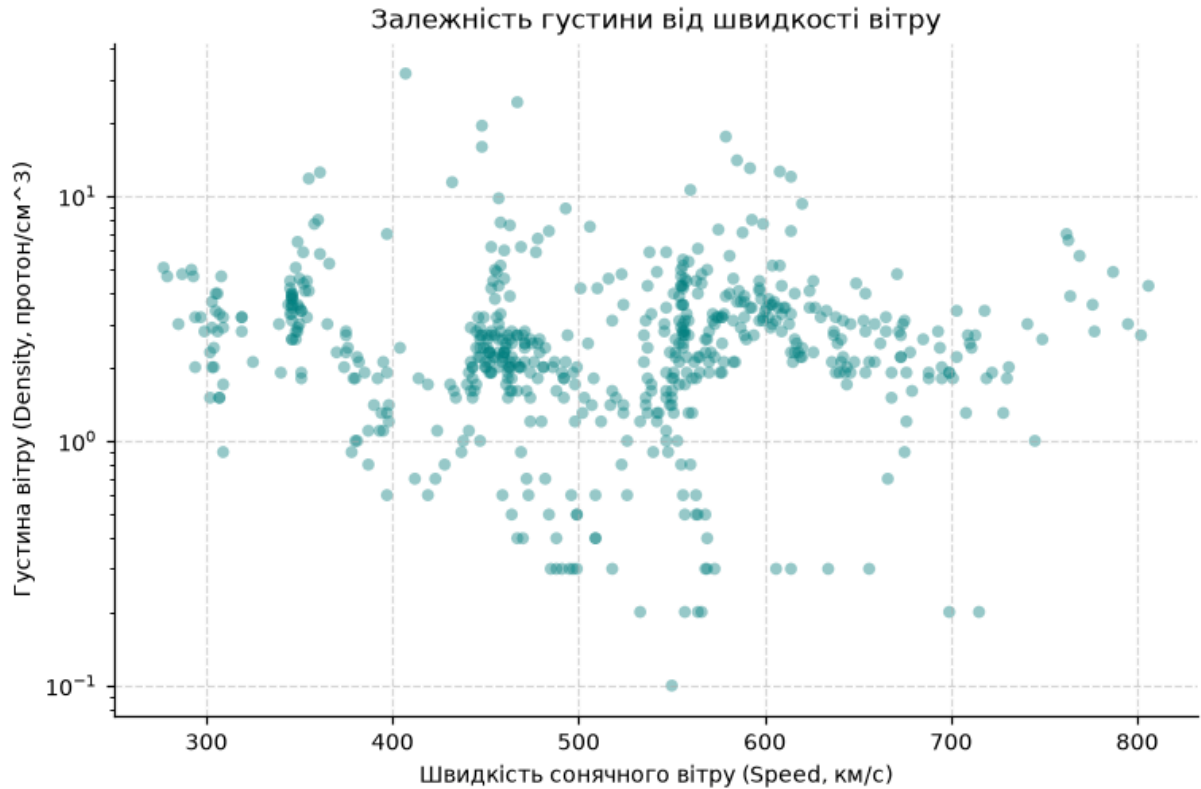
plt.figure(figsize=(8, 5))
plt.scatter(
    M['Speed'],
    M['Density'],
    alpha=0.4,
    color='teal',
    edgecolors='none',
    s=25,
)

plt.yscale('log')

plt.title(
    f'Залежність густини від швидкості вітру'
)
plt.xlabel('Швидкість сонячного вітру (Speed, км/с)')
plt.ylabel('Густина вітру (Density, протон/см^3)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Коефіцієнт кореляції Пірсона: -0.039

Коефіцієнт кореляції Спірмена: 0.016



Висновок: Обидва коефіцієнти кореляції фактично дорівнюють нулю, тобто немає будь-якої лінійної або монотонної залежності між швидкістю сонячного вітру та його густиною

Графік 4. Скільки годин у кожній групі?

Побудуйте стовпчикову діаграму кількості годин за групами. Підпишіть кожен стовпчик кількістю годин і відсотком від загальної кількості.

```
In [22]: # ВАШ КОД
counts = M['група'].value_counts().reindex(['тиха', 'активна', 'дуже активна'])
total = len(M['група'].dropna())
percentages = (counts / total) * 100

plt.figure(figsize=(8, 5))
bars = plt.bar(
    counts.index,
    counts.values,
    color=['blue', 'green', 'red'],
    edgecolor='black',
)

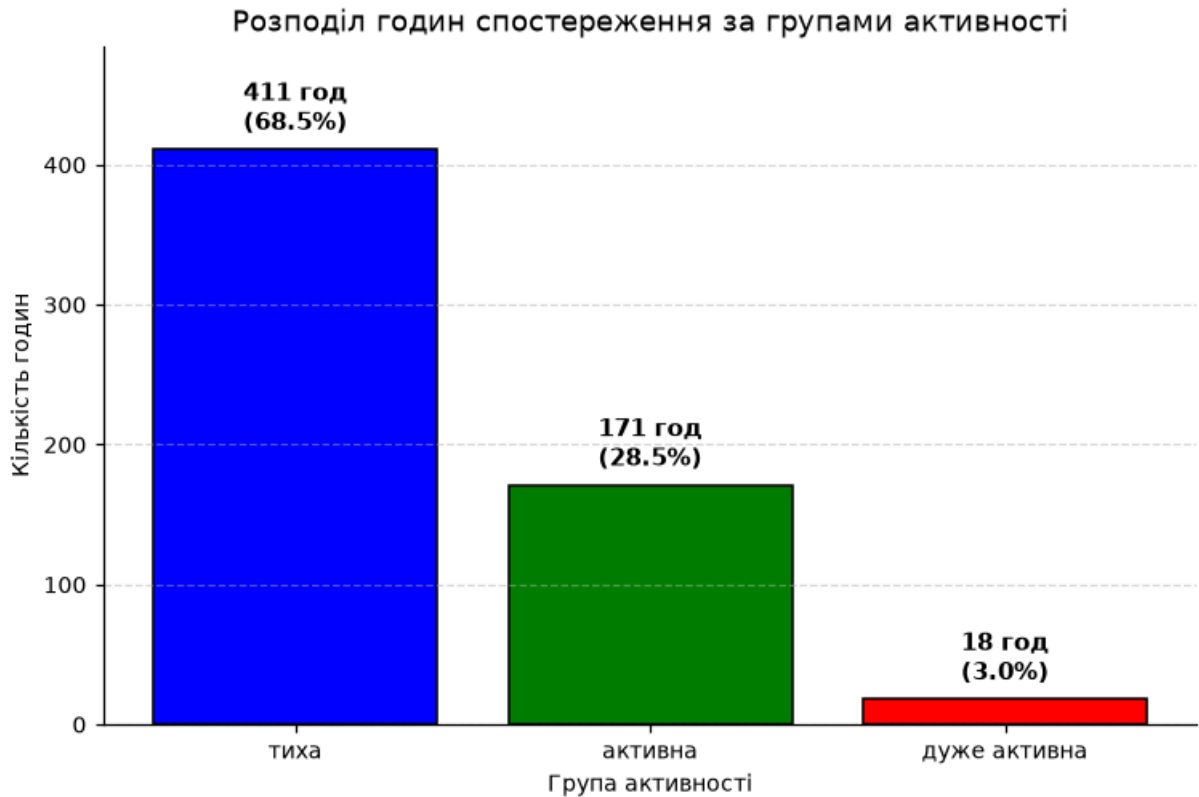
for bar, count, pct in zip(bars, counts.values, percentages.values):
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2.0,
        height + (max(counts.values) * 0.02),
        f'{count} год\n({pct:.1f}%)',
```

```

        ha='center',
        va='bottom',
        fontsize=10,
        fontweight='bold',
    )

plt.title('Розподіл годин спостереження за групами активності', fontsize=12)
plt.xlabel('Група активності')
plt.ylabel('Кількість годин')
plt.ylim(0, max(counts.values) * 1.18)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()

```



Висновок: Більшість часу протони не сильно активні (трохи більше як 2/3 часу), що активніша група, то менше часу вона відбувалась

Графік 5. Як усе це виглядало в часі?

Побудуйте лінійний графік максимуму потоку $P > 10$ за годину за весь період спостережень.

Якщо крива виявиться нечитабельною — згадайте, що допомогло в графіку 1.

```

In [23]: # ВАШ КОД
M_sorted = M.sort_values('ts')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(
    M_sorted['ts'],

```

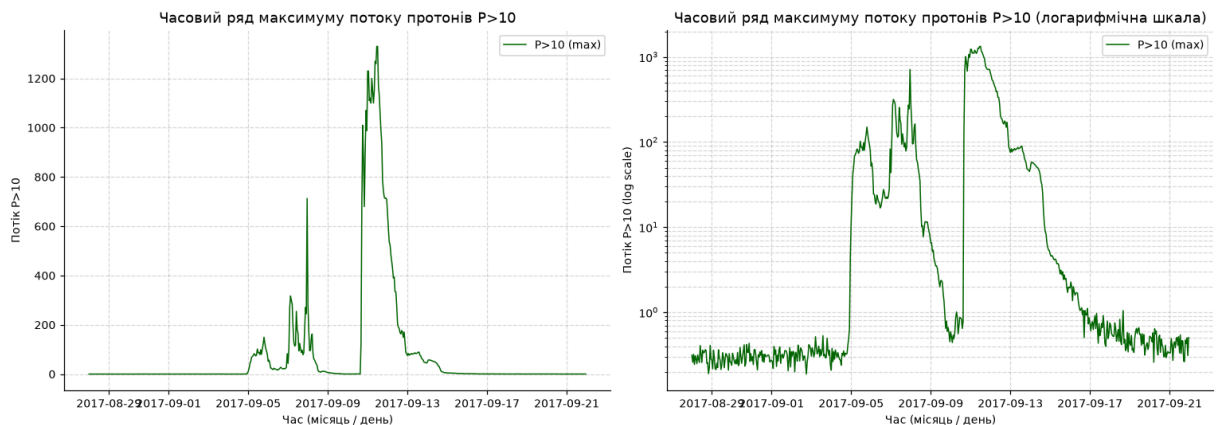
```

M_sorted['P>10_max'],
color='darkgreen',
linewidth=1,
label='P>10 (max)',
)
axes[0].set_title(
    'Часовий ряд максимуму потоку протонів P>10',
    fontsize=12,
)
axes[0].set_xlabel('Час (місяць / день)')
axes[0].set_ylabel('Потік P>10')
axes[0].grid(True, which='both', linestyle='--', alpha=0.5)
axes[0].legend(loc='upper right')

axes[1].plot(
    M_sorted['ts'],
    M_sorted['P>10_max'],
    color='darkgreen',
    linewidth=1,
    label='P>10 (max)',
)
axes[1].set_yscale('log')
axes[1].set_title(
    'Часовий ряд максимуму потоку протонів P>10 (логарифмічна шкала)',
    fontsize=12,
)
axes[1].set_xlabel('Час (місяць / день)')
axes[1].set_ylabel('Потік P>10 (log scale)')
axes[1].grid(True, which='both', linestyle='--', alpha=0.5)
axes[1].legend(loc='upper right')

plt.tight_layout()
plt.show()

```



Висновок: На лінійній шкалі понад 90% часу потік перебуває на рівні близько 0, через що графік стає нечитабельним. Логарифмічна шкала дозволяє одночасно бачити як низький спокійний фон, так і деталі потужних сонячних подій. Переважну більшість часу потік залишається низьким, але під час подій він різко зростає за короткий час і утримується у підвищеному стані кількома суцільними блоками

Етап 6. Два графіки одних і тих самих даних

Нижче побудовано два графіки. Дані в них **одні й ті самі** — це той самий стовпець з тієї самої таблиці, жодне число не змінено.

Виконайте клітинку й подивіться на обидва.

```
In [24]: СТОВПЕЦЬ = 'P>10_max' # <- впишіть назву свого стовпця з МАКСИМУМОМ P>10 за годин

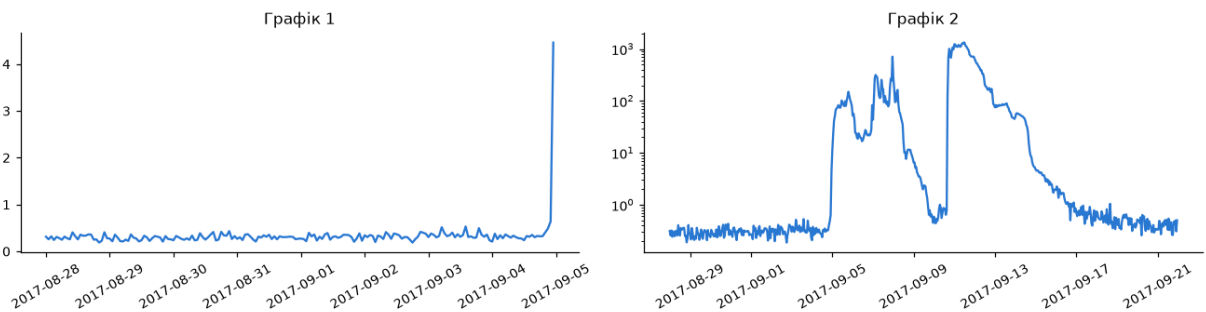
вікно = M[(M['ts'] >= '2017-08-28') & (M['ts'] < '2017-09-05')]

fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))

ax[0].plot(вікно['ts'], вікно[СТОВПЕЦЬ], color='#2a78d6')
ax[0].set_title('Графік 1')
ax[0].tick_params(axis='x', rotation=30)

ax[1].plot(M['ts'], M[СТОВПЕЦЬ], color='#2a78d6')
ax[1].set_yscale('log')
ax[1].set_title('Графік 2')
ax[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```



Завдання 6.1

Дайте відповідь на три питання:

1. Який висновок про космічну погоду у вересні 2017 року зробить людина, яка бачила **тільки перший** графік?
2. Який висновок зробить людина, яка бачила **тільки другий**?
3. Чим ці два графіки відрізняються між собою? Назвіть **дві** відмінності — вони обидві є в коді вище.

Жодне число в обох графіках не підроблене. Різниця виникла тільки через те, як їх побудували.

Відповідь 6.1:

1. Висновок з першого графіка:
2. Висновок з другого графіка:
3. Дві відмінності між ними:

Завдання 6.2

Уявіть, що вам потрібно **одним графіком** чесно показати, що відбувалося в цьому періоді. Побудуйте його самі й підпишіть заголовком, який формулює висновок.

Поясніть у клітинці нижче, чому ви обрали саме такий вигляд.

```
In [25]: # ВАШ КОД
M_sorted = M.sort_values('ts')

plt.figure(figsize=(12, 5))
plt.plot(
    M_sorted['ts'],
    M_sorted['P>10_max'],
    color='darkgreen',
    linewidth=1.5,
    label='Потік P>10 (max)',
)

plt.yscale('log')

plt.axhline(
    10,
    color='orange',
    linestyle='--',
    alpha=0.7,
    label='Група "активна" (10)'
)

plt.axhline(
    1000,
    color='red',
    linestyle='--',
    alpha=0.7,
    label='Група "дуже активна" (1000)',
)

plt.title(
    'Вересень 2017: Два шторми зі зростом потоку у більше 1000 разів за норму',
    fontsize=11,
    fontweight='bold',
)

plt.xlabel('Дата')
plt.ylabel('Потік частинок (логарифмічна шкала)')
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.legend(loc='upper right')
```

```
plt.tight_layout()
plt.show()
```



Відповідь 6.2: Даний графік показує увесь часовий період та використовує логарифмування для того, щоб не губити ані дрібних деталей, ані екстремумів

Етап 7. Три спостереження та ваше питання

Завдання 7.1

Запишіть **три спостереження** про ці дані, які ви зробили під час роботи.

Спостереження — це твердження, яке спирається на конкретне число або графік з вашого ноутбука. Не «дані цікаві», а, наприклад, «максимальне значення потоку у 2000 разів більше за медіанне, і всі такі години припадають на одні й ті самі три доби».

Для кожного вкажіть, **де саме** у вашій роботі його видно.

Спостереження 1: У стовпці E>2.0_mean середнє значення (11397) перевищує медіану (4338.8) майже в 2.6 рази. Це доводить, що більшість часу потік перебуває в спокійному стані з низькими значеннями, а середнє значення сильно витягнуте вгору невеликою кількістю екстремальних спалахів

Спостереження 2: Більш ніж 2/3 всього часу спостережень (~70%) припадає на групу «тиха» ($P > 10_{\text{max}} < 10$), тоді як події групи «дуже активна» трапляються рідше ніж у 5% годин. Виплески протонів є рідкісними імпульсними подіями, а не стаціонарним шумом

Спостереження 3: Коефіцієнти кореляції Пірсона ($r = -0.039$) та Спірмена ($\rho = 0.016$) між густиною та швидкістю протонів фактично дорівнюють 0. Це підтверджує, що густина та швидкість вітру є повністю незалежними фізичними параметрами

Завдання 7.2 — ваше індивідуальне питання

Дайте відповідь на додаткове питання зі свого завдання. Відповідь має спиратися на обчислення: наведіть код і результат, а потім сформулюйте висновок словами.

```
In [26]: # ВАШ КОД
time_cols = [
    'Year',
    'Month',
    'Day',
    'Hour',
    'Minute',
    'Julian date',
    'Seconds in day',
]
numeric_df = M.select_dtypes(include=['number']).drop(
    columns=time_cols, errors='ignore'
)

cv_results = (numeric_df.std() / numeric_df.mean()).abs()

cv_sorted = cv_results.sort_values(ascending=False)

print('Коефіцієнт варіації (медіана / середнє) для всіх стовпців:')
print(cv_sorted)

print(
    f'\nНайбільший розкид має стовпець: "{cv_sorted.index[0]}" (CV = {cv_sorted.iloc[0]})
    )
```

Коефіцієнт варіації (медіана / середнє) для всіх стовпців:

P>100_mean	4.308374
P>100_max	4.223354
P>50_mean	3.842017
P>50_max	3.778265
P>30_mean	3.569527
P>30_max	3.510205
P>10_mean	2.901722
P>10_max	2.826926
P>1_max	2.654705
P>5_mean	2.476875
P>5_max	2.441740
P>1_mean	2.167933
E>2.0_mean	1.474061
E>2.0_max	1.423943
Density	0.883661
E>0.8_mean	0.799040
E>0.8_max	0.759051
Radio Flux 10.7	0.239608
Speed	0.216257

dtype: float64

Найбільший розкид має стовпець: "P>100_mean" (CV = 4.31)

Відповідь 7.2: Найбільший розкид має стовпець P>100_mean, як один з високоенергетичних потоків, де дві мінімальні зміни мінімального та максимального можуть дати велику зміну середнього

Етап 8. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями під час виконання цієї роботи **дозволено**. Це нормальна інженерна практика, і курс про штучний інтелект не має підстав її забороняти.

Обов'язковою є прозорість: ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення.

Задекларована допомога порушенням не є.

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок вашого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці. Якщо код згенеровано, але ви його розумієте — робота зарахована. Якщо не розумієте — не зарахована, незалежно від того, хто його написав.

Завдання 8.1

Заповніть словник. Для кожного етапу оберіть одне з формулювань:

- 'ні' — інструментами ШІ не користувався;
- 'синтаксис' — питав про назву функції або синтаксис, код писав сам;
- 'код, перевірено' — код згенеровано, я його прочитав, перевірів і розумію;
- 'текст, перероблено' — згенеровано чернетку тексту, яку я переписав по суті;
- 'текст, як є' — згенерований текст залишено майже без змін.

Останній варіант не заборонений, але прямо впливає на оцінку: бали ставляться за висновки, а не за наявність абзацу.

```
In [27]: ІНСТРУМЕНТИ_ШІ = 'Gemini'      # напр. 'ChatGPT' або 'не використовував'

ДЕКЛАРАЦІЯ = {
    'етапи 1-2, читання файлу та розбір таблиць': 'ні',
    'етап 3, паспорт даних і дивні значення':      'ні',
    'етап 4, об'єднання таблиць':                  'синтаксис',
    'етап 5, побудова графіків':                    'код, перевірено',
    'етап 5, формулювання висновків':              'текст, перероблено',
    'етапи 6-7, порівняння графіків і спостереження': 'текст, перероблено',
}

# Що саме ви перевірили за згенерованим кодом чи текстом — конкретно:
# які числа звірили, що переписали, де знайшли помилку.
ЩО_ПЕРЕВІРЕНО = 'при створенні таблиці A_hourly не було додано усі попередні стовпці
'У завданні 4.2 було використано два методи, де мінімальне вибиралося з таблиці A,
'У графіках завдання 5 було змінено кольори, назви та деякі деталі'

ПІДТВЕРДЖУЮ = True
```

```
In [28]: # --- перевірка повноти декларації: клітинку не редагувати -----
ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'текст, як

assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не використовував
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()]
assert not порожні, 'Не заповнено пункти декларації: %s' % '; '.join(порожні)
чужі = {v for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ}
assert not чужі, 'Недопустимі значення: %s. Оберіть з переліку вище.' % ', '.join(s
assert len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше, що саме ви переві
          '(зараз %d символів, потрібно щонайменше
          % len(ЩО_ПЕРЕВІРЕНО.strip()))
assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що можете пояснити власний код на захисті

print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78)
print('%-46s %s' % ('автор:', ПІБ))
print('%-46s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78)
for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-46s %s' % (k + ':', v))
print('-' * 78)
print('перевірено автором:')
print(ЩО_ПЕРЕВІРЕНО)
print('=' * 78)
print('Автор підтверджує, що може пояснити кожен рядок коду під час захисту.')
```

ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ

```

=====
автор:                                Бугай Софія
інструменти:                          Gemini
-----

етапи 1-2, читання файлу та розбір таблиць:    ні
етап 3, паспорт даних і дивні значення:        ні
етап 4, об'єднання таблиць:                    синтаксис
етап 5, побудова графіків:                      код, перевірено
етап 5, формулювання висновків:                 текст, перероблено
етапи 6-7, порівняння графіків і спостереження: текст, перероблено
-----

перевірено автором:
при створенні таблиці A_hourly не було додано усі попередні стовпці, які були важливі
у наступних етапах
У завданні 4.2 було використано два методи, де мінімальне вибиралося з таблиці A, а
середнє з таблиці A_hourly
У графіках завдання 5 було змінено кольори, назви та деякі деталі
=====
Автор підтверджує, що може пояснити кожен рядок коду під час захисту.

```

Крок 9. Збереження результату та фінальна самоперевірка

Об'єднана таблиця знадобиться вам у лабораторних роботах № 3 і № 4 — збережіть її й не втрачайте до кінця семестру.

```
In [29]: M.to_csv('master_hourly.csv', index=False)
print('збережено:', M.shape)
```

збережено: (601, 25)

```
In [ ]: # --- фінальна самоперевірка -----
перевірки = {
    'ПІБ і варіант заповнено':    bool(ПІБ) and bool(ВАРІАНТ),
    'таблиця A (7200 рядків)':    len(A) == 7200,
    'таблиця B (25 рядків)':      len(B) == 25,
    'таблиця C (601 рядок)':      len(C) == 601,
    'таблиці об'єднано':          M is not None and 590 <= len(M) <= 605,
    'стовпець «група» є':         'група' in M.columns,
    'усі дати у 2017 році':       int(pd.to_datetime(M['ts']).dt.year.min()) ==
    'декларацію заповнено':       bool(ІНСТРУМЕНТИ_ШІ.strip()) and ПІДТВЕРДЖУЮ
}
for k, v in перевірки.items():
    print(('OK' if v else 'НЕ OK') + ' ' + k)

if all(перевірки.values()):
    print('\nТехнічна частина роботи виконана.')
    print('Тепер переконайтеся, що заповнені ВСІ клітинки з відповідями та висновка')
    print('і виконайте Kernel → Restart & Run All перед здачею.')
```

OK ПІБ і варіант заповнено
OK таблиця А (7200 рядків)
OK таблиця В (25 рядків)
OK таблиця С (601 рядок)
OK таблиці об'єднано
OK стовпець «група» є
OK усі дати у 2017 році
OK декларацію заповнено

Технічна частина роботи виконана.

Тепер переконайтеся, що заповнені ВСІ клітинки з відповідями та висновками,
і виконайте Kernel → Restart & Run All перед здачею.